

ADITYA'S INDIVIDUAL TASK SHEET

Role: Team + Senior Backend Developer + AI Integration Specialist

Team: Team A (Backend & AI) - Partner: Rushikesh

Timeline: 3 Months (90 Days)

Total Hours: 864 hours (72 hours/week × 12 weeks)

Working Days: Monday to Saturday (12 hours/day)

Rest Day: Sunday (Mandatory - NO CODING!)

YOUR PROFILE & RESPONSIBILITIES

Primary Focus Areas:

1. **Architecture & Technical Leadership** (20% time)

- System design and architecture decisions
- Database schema design
- API structure and patterns
- Code reviews and quality standards
- Team coordination and blocker resolution

2. **Backend Development** (50% time)

- Core APIs (Auth, Questions, Tests, Exams)
- Business logic implementation
- Database optimization
- Payment integration
- Analytics and reports

3. **AI Integration** (20% time)

- OpenAI/Claude API integration
- Question classification AI
- Similar question generation
- OCR integration

4. **DevOps & Deployment** (10% time)

- Production setup
- Monitoring and error tracking
- Performance optimization

Your Strengths (Use These):

- Strong backend architecture skills

- Problem-solving and algorithm design
- AI/ML integration experience
- Leadership and decision-making
- Quick learning ability

What You'll Learn:

- Supabase advanced features
- AI prompt engineering at scale
- Production-grade system design
- Team leadership under pressure



MONTH 1: FOUNDATION (WEEKS 1-4)

WEEK 1: SETUP & AUTHENTICATION (72 hours)

Your Focus: 40 hours implementation + 32 hours leadership/coordination

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Project Foundation

Hour-by-Hour Breakdown:

- **Hours 1-4:** Repository and project setup
 - ☐ Create GitHub organization/repo with proper structure
 - ☐ Set up mono-repo (using Turborepo or Nx if needed)
- ☐ Create folder structure:
/apps /web (Next.js) /mobile (React Native) /api (NestJS)/packages /ui (shared components) /utils (shared utilities)/docs
 - ☐
 - ☐ Initialize Git workflow (main, develop, feature branches)
 - ☐ Set up commit conventions (Conventional Commits)
- **Hours 5-8:** Supabase initialization
 - ☐ Create Supabase project (free tier)
 - ☐ Set up development and staging databases
 - ☐ Configure Supabase CLI locally
 - ☐ Install Supabase libraries
- **Hours 9-12:** Database ER design
 - ☐ Design complete ER diagram (use dbdiagram.io)

[] Tables to create:

- institutes (id, name, subscription_plan, settings, created_at)- users (id, institute_id, role, name, email, phone, password_hash)- students (id, user_id, class, batch, face_encoding)- teachers (id, user_id, subjects, classes)- parents (id, user_id, student_ids)- subjects (id, name, code)- chapters (id, subject_id, name, order)- topics (id, chapter_id, name)

-
- [] Review with team (30-minute sync)
- **Hours 13-16:** Create initial tables
 - [] Write SQL migration files
 - [] Create tables in Supabase
 - [] Set up Row Level Security (RLS) policies
 - [] Test basic CRUD with Supabase client
- **Hours 17-20:** Team coordination
 - [] Share database schema with Atharav & Umesh
 - [] Set up Postman workspace (shared collection)
 - [] Write initial API documentation template
 - [] Help Rushikesh with local setup
- **Hours 21-24:** Planning & documentation
 - [] Write README for backend project
 - [] Document API naming conventions
 - [] Create .env.example file
 - [] Code review guidelines document

Deliverables Day 1-2:

- ☒ GitHub repo with structure
 - ☒ Supabase project initialized
 - ☒ Database schema designed and created
 - ☒ Team has access and documentation
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Authentication System

Hour-by-Hour Breakdown:

- **Hours 1-6:** Supabase Auth setup
 - [] Configure Supabase Auth settings
 - [] Enable email/password authentication
 - [] Set up JWT configuration
 - [] Test authentication flow in Supabase dashboard
 - [] Create custom auth hooks

- **Hours 7-12:** Role-Based Access Control (RBAC)

- ☐ Define roles enum (ADMIN, TEACHER, STUDENT, PARENT)
- ☐ Create role-based policies in Supabase
- ☐ Write middleware for role checking

☐ Implement permission matrix:

ADMIN: full access
TEACHER: manage classes, tests, view students
STUDENT: take tests, view own data
PARENT: view child's data only

○

- **Hours 13-18:** Core auth APIs

- ☐ POST /auth/register (with role selection)
- ☐ POST /auth/login (returns JWT + user data)
- ☐ POST /auth/logout
- ☐ GET /auth/me (get current user)
- ☐ POST /auth/refresh-token
- ☐ All APIs tested in Postman

- **Hours 19-24:** Advanced auth features

- ☐ Email verification logic
- ☐ Password strength validation
- ☐ Rate limiting on login (prevent brute force)
- ☐ Session management
- ☐ Write auth unit tests
- ☐ Code review with Rushikesh

Deliverables Day 3-4:

- ☒ Authentication system fully functional
 - ☒ RBAC implemented
 - ☒ 5+ auth APIs tested
 - ☒ Postman collection shared with team
-

Day 5 (Friday) - 12 hours

Task: API Structure & Documentation

- **Hours 1-4:** API architecture

- ☐ Define RESTful API naming conventions

☐ Create base response format:

```
{ "success": true, "data": {}, "message": "Success", "timestamp":  
"2026-01-28T10:00:00Z"}
```

-
- ☐ Error handling middleware
- ☐ Logging setup (Winston or Pino)
- **Hours 5-8: Documentation setup**
 - ☐ Set up Swagger/OpenAPI
 - ☐ Document auth APIs
 - ☐ Create Postman collection template
 - ☐ API versioning strategy (v1/...)
- **Hours 9-12: Team sync & planning**
 - ☐ Demo auth system to Team B
 - ☐ Review Week 2 tasks with Rushikesh
 - ☐ Unblock Atharav/Umesh if stuck
 - ☐ Update project board (GitHub Projects)

Deliverables Day 5:

- ☒ API documentation live
 - ☒ Standards and conventions documented
 - ☒ Team aligned for Week 2
-

Day 6 (Saturday) - 12 hours

Task: Support & Review

- **Hours 1-6: Code reviews**
 - ☐ Review Rushikesh's registration APIs
 - ☐ Review Atharav's UI code (if needed)
 - ☐ Ensure code quality standards
- **Hours 7-12: Preparation for Week 2**
 - ☐ Research question bank design patterns
 - ☐ Study CSV parsing libraries (PapaParse, xlsx)
 - ☐ Design question schema in detail
 - ☐ Create Week 2 task breakdown

Week 1 Complete! 🎉

- Total Hours: 40 hours direct work + 32 hours coordination
 - Team Integration Point: Auth APIs ready for frontend ☒
-

WEEK 2: QUESTION BANK SYSTEM (72 hours)

Your Focus: 36 hours implementation + 36 hours architecture/review

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Question Database Design

- **Hours 1-6:** Schema design

[] Design questions table:

```
CREATE TABLE questions ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
institute_id UUID REFERENCES institutes(id), subject_id UUID REFERENCES
subjects(id), chapter_id UUID REFERENCES chapters(id), topic_id UUID REFERENCES
topics(id), question_text TEXT NOT NULL, question_type VARCHAR(50), -- MCQ,
NUMERICAL, SUBJECTIVE options JSONB, -- for MCQ: {A, B, C, D} correct_answer
TEXT, explanation TEXT, difficulty VARCHAR(20), -- EASY, MEDIUM, HARD marks
INTEGER DEFAULT 1, negative_marks DECIMAL DEFAULT 0, time_estimate INTEGER,
-- seconds image_url TEXT, tags TEXT[], -- array of tags source VARCHAR(100), --
TEACHER_CREATED, AI_GENERATED, IMPORTED created_by UUID REFERENCES
users(id), created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT
NOW());CREATE INDEX idx_questions_subject ON questions(subject_id);CREATE INDEX
idx_questions_chapter ON questions(chapter_id);CREATE INDEX idx_questions_difficulty
ON questions(difficulty);CREATE INDEX idx_questions_tags ON questions USING
GIN(tags);
```

○

- **Hours 7-12:** CRUD APIs - Create & Read
 - [] POST /api/questions (create single question)
 - [] POST /api/questions/bulk (create multiple)
 - [] GET /api/questions (list with pagination)
 - [] GET /api/questions/:id (get single question)
 - [] Add input validation (Zod schemas)
 - [] Test all endpoints in Postman
- **Hours 13-18:** CRUD APIs - Update & Delete
 - [] PATCH /api/questions/:id (update question)
 - [] DELETE /api/questions/:id (soft delete)
 - [] Add authorization (only creator or admin can edit)
 - [] Version history (track changes)
- **Hours 19-24:** Search & Filter APIs
 - [] GET /api/questions/search
 - [] Query params:
 - subject, chapter, topic
 - difficulty
 - tags (multiple)
 - question_type

- created_by
- ☐ Full-text search on question_text
- ☐ Sort options (newest, difficulty, marks)
- ☐ Pagination (limit, offset)

Deliverables Day 1-2:

- ☒ Question database schema created
 - ☒ 6+ CRUD APIs implemented
 - ☒ Search and filter working
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Bulk Upload & Image Handling

- **Hours 1-8:** CSV/Excel upload processing
 - ☐ POST /api/questions/import
 - ☐ Accept file upload (multer middleware)
 - ☐ Parse CSV (PapaParse library)
 - ☐ Parse Excel (xlsx library)
 - ☐ Validate each row:
 - Required fields present
 - Data types correct
 - Subject/chapter exists
 - ☐ Bulk insert into database (batch of 100)

☐ Return import summary:

```
{ "total": 500, "success": 485, "failed": 15, "errors": [...]}
```

○

- **Hours 9-16:** Image handling
 - ☐ Integrate Supabase Storage
 - ☐ POST /api/questions/upload-image
 - ☐ Image validation (size < 5MB, format: jpg/png)
 - ☐ Generate unique filename
 - ☐ Store in Supabase Storage bucket
 - ☐ Return public URL
 - ☐ Attach image URL to question
- **Hours 17-24:** Testing & optimization
 - ☐ Test bulk import with 1000 questions
 - ☐ Test image upload (multiple formats)
 - ☐ Add progress tracking for large imports
 - ☐ Optimize database insert performance
 - ☐ Write API documentation

Deliverables Day 3-4:

- ☒ Bulk import working (CSV + Excel)
 - ☒ Image upload integrated
 - ☒ Can import 1000+ questions quickly
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Question Tagging & Advanced Features

- **Hours 1-8:** Tagging system
 - ☐ Auto-extract tags from question text (simple keyword matching)
 - ☐ GET /api/tags (list all unique tags)
 - ☐ POST /api/questions/:id/tags (add tags)
 - ☐ DELETE /api/questions/:id/tags (remove tags)
 - ☐ Tag-based search optimization
- **Hours 9-16:** Question templates
 - ☐ Create template system for common question types
 - ☐ Example: "Speed = Distance / Time" numerical template
 - ☐ Save templates for reuse
 - ☐ Generate questions from templates
- **Hours 17-24:** Team support & review
 - ☐ Help Rushikesh with testing
 - ☐ Code review (all Week 2 code)
 - ☐ Demo question bank to Team B (Atharav & Umesh)
 - ☐ Prepare sample 100 questions for frontend testing
 - ☐ Week 2 retrospective

Week 2 Complete! 🎉

- Integration Point: Question bank APIs ready for UI ☒
-

WEEK 3: TEST CREATION SYSTEM (72 hours)

Your Focus: 40 hours implementation + 32 hours design/coordination

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Test Database & Configuration APIs

- **Hours 1-8:** Test schema design

[] Create tests table:

```
CREATE TABLE tests ( id UUID PRIMARY KEY, institute_id UUID REFERENCES
institutes(id), created_by UUID REFERENCES users(id), title VARCHAR(255) NOT NULL,
description TEXT, test_type VARCHAR(50), -- PRACTICE, MOCK, CHAPTER, FULL
total_marks INTEGER, duration INTEGER, -- minutes start_time TIMESTAMP, end_time
TIMESTAMP, instructions JSONB, is_published BOOLEAN DEFAULT false,
proctoring_enabled BOOLEAN DEFAULT false, created_at TIMESTAMP DEFAULT
NOW());CREATE TABLE test_sections ( id UUID PRIMARY KEY, test_id UUID
REFERENCES tests(id), name VARCHAR(100), -- Physics, Chemistry, Math duration
INTEGER, -- minutes (if sectional timing) marks INTEGER, section_order
INTEGER);CREATE TABLE test_questions ( id UUID PRIMARY KEY, test_id UUID
REFERENCES tests(id), section_id UUID REFERENCES test_sections(id), question_id
UUID REFERENCES questions(id), marks INTEGER, negative_marks DECIMAL,
question_order INTEGER);
```

○

- **Hours 9-16:** Test CRUD APIs
 - [] POST /api/tests (create test)
 - [] GET /api/tests (list tests)
 - [] GET /api/tests/:id (get single test)
 - [] PATCH /api/tests/:id (update test)
 - [] DELETE /api/tests/:id (delete test)
 - [] POST /api/tests/:id/publish (publish test)
- **Hours 17-24:** Section management
 - [] POST /api/tests/:testId/sections (add section)
 - [] PATCH /api/tests/:testId/sections/:sectionId
 - [] DELETE /api/tests/:testId/sections/:sectionId
 - [] GET /api/tests/:testId/sections (list sections)

Deliverables Day 1-2:

-  Test database schema created
 -  Basic test CRUD APIs working
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Question Selection & Assignment

- **Hours 1-10:** Question selection logic
 - [] POST /api/tests/:testId/questions (add questions to test)
 - [] Support multiple selection methods:
 - Manual: Select specific question IDs
 - Filter-based: Select by chapter, difficulty, tags
 - Random: Pick N questions matching criteria

- [] Validate total marks match test configuration
- [] Prevent duplicate questions
- **Hours 11-18:** Student assignment

[] Create test_assignments table:

```
CREATE TABLE test_assignments ( id UUID PRIMARY KEY, test_id UUID REFERENCES tests(id), student_id UUID REFERENCES users(id), assigned_at TIMESTAMP DEFAULT NOW(), status VARCHAR(50) -- ASSIGNED, IN_PROGRESS, COMPLETED);
```

- [] POST /api/tests/:testId/assign (assign to students)
- [] Support batch assignment (entire class/batch)
- [] GET /api/students/:studentId/tests (student's assigned tests)
- **Hours 19-24:** Test scheduling
 - [] Validate start_time < end_time
 - [] Automatic status updates (scheduled, live, completed)
 - [] Scheduled notifications (remind students 1 hour before)
 - [] Test preview API (for teachers to review before publishing)

Deliverables Day 3-4:

-  Question selection working
-  Student assignment APIs ready

Day 5-6 (Friday-Saturday) - 24 hours

Task: Test Templates & Advanced Features

- **Hours 1-12:** Test templates
 - [] Save test configuration as template
 - [] POST /api/test-templates (create template)
 - [] GET /api/test-templates (list templates)
 - [] POST /api/tests/from-template/:templateId (create test from template)
 - [] Common templates: JEE Main, NEET, Chapter Test
- **Hours 13-20:** Marks distribution
 - [] Auto-calculate total marks when questions added
 - [] Validate marks per section
 - [] Marks distribution report for teachers
- **Hours 21-24:** Code review & demo
 - [] Review all Week 3 code
 - [] Demo test creation flow to Team B
 - [] Update Postman collection

- [] Week 3 retrospective

Week 3 Complete! 🎉

- Integration Point: Teachers can create tests ✓

WEEK 4: TEST SESSION & AUTO-GRADING (72 hours)

Your Focus: 40 hours implementation + 32 hours testing/optimization

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Test Session Management

- **Hours 1-10:** Session tracking

[] Create test_attempts table:

```
CREATE TABLE test_attempts ( id UUID PRIMARY KEY, test_id UUID REFERENCES tests(id), student_id UUID REFERENCES users(id), started_at TIMESTAMP, ended_at TIMESTAMP, time_taken INTEGER, -- seconds status VARCHAR(50), -- IN_PROGRESS, COMPLETED, SUBMITTED score INTEGER, percentage DECIMAL, rank INTEGER);
CREATE TABLE student_responses ( id UUID PRIMARY KEY, attempt_id UUID REFERENCES test_attempts(id), question_id UUID REFERENCES questions(id), answer TEXT, is_correct BOOLEAN, marks_awarded DECIMAL, time_taken INTEGER, -- seconds per question marked_for_review BOOLEAN, answered_at TIMESTAMP);
```

-
- **Hours 11-18:** Test start/submit APIs
 - [] POST /api/tests/:testId/start (start test session)
 - Validate: test is live, student is assigned, not already attempted
 - Create test_attempt record
 - Return test questions (randomized order if configured)
 - [] POST /api/tests/attempts/:attemptId/submit (submit test)
 - Validate all required questions answered
 - Mark attempt as completed
 - Trigger auto-grading
- **Hours 19-24:** Answer submission
 - [] POST /api/tests/attempts/:attemptId/answer (save single answer)
 - [] Auto-save every 30 seconds (from frontend)
 - [] Support answer updates (until test submitted)
 - [] Mark for review functionality

Deliverables Day 1-2:

- ☒ Test session management working
 - ☒ Students can start and submit tests
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Auto-Grading Engine

- **Hours 1-12:** MCQ auto-grading

[] Build grading engine:

```
async function gradeAttempt(attemptId) { // Get all responses for this attempt // For each
MCQ: // - Compare student answer with correct answer // - Award marks if correct // -
Deduct negative marks if wrong // Calculate total score // Update test_attempt record //
Calculate rank (compared to other students)}
```

-
- [] Handle different question types:
 - Single correct MCQ
 - Multiple correct MCQ (all must be selected)
 - Numerical (exact match or tolerance range)
- [] Negative marking logic
- **Hours 13-20:** Results calculation
 - [] Calculate total score
 - [] Calculate percentage
 - [] Calculate rank (compared to other students who attempted)
 - [] Section-wise breakdown
 - [] Time analysis (fast/slow questions)
- **Hours 21-24:** Results API
 - [] GET /api/tests/attempts/:attemptId/results

[] Return detailed breakdown:

```
{ "score": 285, "totalMarks": 300, "percentage": 95, "rank": 2, "totalAttempts": 150,
"sectionWise": [...], "timeAnalysis": {...}, "correctQuestions": 95, "incorrectQuestions": 5,
"unattempted": 0}
```

○

Deliverables Day 3-4:

- ☒ Auto-grading working for MCQs
 - ☒ Results calculated instantly
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Analytics & Optimization

- **Hours 1-12:** Test analytics
 - ☐ GET /api/tests/:testId/analytics
 - ☐ Calculate for entire test:
 - Average score
 - Highest/lowest score
 - Pass percentage
 - Question-wise accuracy
 - Time-wise distribution
 - ☐ Identify difficult questions (< 30% accuracy)
- **Hours 13-18:** Attempt tracking
 - ☐ Track time per question
 - ☐ Prevent multiple attempts (if configured)
 - ☐ Auto-submit when time expires (cron job)
- **Hours 19-24:** Month 1 wrap-up
 - ☐ Performance testing (100 concurrent users)
 - ☐ Database optimization (add missing indexes)
 - ☐ Code cleanup and refactoring
 - ☐ Month 1 demo to team
 - ☐ Plan Month 2 tasks

Week 4 Complete! Month 1 Done! 🎉🎉

- Integration Point: Full test cycle working (create → take → grade → results) 



MONTH 2: AI FEATURES & FACE RECOGNITION (WEEKS 5-8)

WEEK 5: AI QUESTION CLASSIFICATION & GENERATION (72 hours)

YOUR MOST CRITICAL WEEK - This is where you shine!

Your Focus: 50 hours AI work + 22 hours coordination

Day 0 (Pre-Week Research - Sunday Evening 4 hours)

Preparation is key for AI week!

- ☐ Read OpenAI API documentation (1 hour)
- ☐ Read Claude API documentation (1 hour)

- ☐ Study prompt engineering best practices (1 hour)
- ☐ Test basic API calls (sample code) (1 hour)

Day 1-2 (Monday-Tuesday) - 24 hours

Task: AI Integration Foundation

- **Hours 1-6:** API setup & selection
 - ☐ Research costs:
 - OpenAI GPT-4 Turbo: ~\$10 per 1M tokens input
 - Claude Sonnet: ~\$3 per 1M tokens
 - ☐ Decision: Start with Claude (cheaper), fallback to OpenAI
 - ☐ Set up API keys in environment variables

☐ Create AI service wrapper:

```
class AIService {
  async classify(questionText) { }
  async generateSimilar(questionText,
    concept, difficulty) { }}
```

- ☐ Implement rate limiting (avoid API quota exceeded)
- **Hours 7-16:** Question classification

☐ Design classification prompt:

You are an expert educational content classifier for JEE/NEET exams.

Analyze this question and extract:

1. Subject (Physics/Chemistry/Math/Biology)
2. Chapter (e.g., "Mechanics", "Thermodynamics")
3. Topic (e.g., "Kinematics", "Laws of Motion")
4. Sub-topic (e.g., "Projectile Motion", "Work-Energy Theorem")
5. Difficulty (Easy/Medium/Hard) - based on:
 - Conceptual complexity
 - Number of steps to solve
 - Formula complexity
6. Concepts involved (array of key concepts/formulas)
7. Suggested tags (5-10 keywords)

Question: {questionText}

Return response as JSON only (no markdown):

```
{
  "subject": "",
  "chapter": "",
  "topic": "",
  "subTopic": "",
  "difficulty": "",
```

```
"concepts": [],  
"tags": []  
}
```

- - ☐ Test classification with 10 sample questions
 - ☐ Validate JSON response parsing
 - ☐ Handle API errors gracefully
- **Hours 17-24: Classification API**
 - ☐ POST /api/ai/classify-question
 - ☐ Accept question text
 - ☐ Call AI service
 - ☐ Update question record with classification
 - ☐ Cache results (avoid re-classification)
 - ☐ Batch processing API (classify 100 questions at once)

Deliverables Day 1-2:

- ☒ AI service integrated
 - ☒ Question classification working
 - ☒ Tested with sample questions
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Similar Question Generation

- **Hours 1-12: Generation prompt engineering**

☐ Design generation prompt:

You are an expert question generator for JEE/NEET preparation.

Original Question: {originalQuestion}

Correct Answer: {answer}

Concepts: {concepts}

Difficulty: {difficulty}

Generate 5 similar questions that:

1. Test the SAME concept/formula
2. Have the SAME difficulty level
3. Use DIFFERENT scenarios/contexts
4. Use DIFFERENT numerical values (if applicable)

5. Are clear and unambiguous

For EACH question, provide:

- Question text (complete, ready to use)
- Options (A, B, C, D) if MCQ
- Correct answer
- Brief solution steps

Return as JSON array:

```
[
  {
    "questionText": "",
    "options": {"A": "", "B": "", "C": "", "D": ""},
    "correctAnswer": "",
    "solution": ""
  },
  ...
]
```

-
- [] Test with 5 different question types:
 - Numerical (kinematics)
 - Conceptual (thermodynamics)
 - Formula-based (electricity)
 - Problem-solving (mechanics)
 - Theory (biology)
- **Hours 13-20:** Template-based generation (fallback)

[] For numerical questions, create template system:

// Example: Speed = Distance / Time
const template = { pattern: "{object} travels {distance} km in {time} hours. Find speed.", variables: { object: ["car", "bike", "train", "bus"], distance: [60, 80, 120, 150], time: [2, 3, 4, 1.5] }, formula: "speed = distance / time";

-
- [] Generate variations by substituting values
- [] Ensure answer correctness
- **Hours 21-24:** Generation API
 - [] POST /api/ai/generate-similar
 - [] Input: questionId
 - [] Process:
 - Get original question details
 - Call AI service (or use template if numerical)
 - Validate generated questions
 - Save to database (mark as AI_GENERATED)
 - [] Return 5 similar questions

Deliverables Day 3-4:

- ☒ Similar question generation working
 - ☒ Both AI and template-based methods implemented
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Adaptive Learning Integration

- **Hours 1-12:** Adaptive learning logic

[] Create learning_paths table:

```
CREATE TABLE learning_paths (  
  id UUID PRIMARY KEY,  
  student_id UUID REFERENCES users(id),  
  concept VARCHAR(200),  
  proficiency VARCHAR(20), -- WEAK, MODERATE, STRONG  
  questions_attempted INTEGER DEFAULT 0,  
  questions_correct INTEGER DEFAULT 0,  
  last_practiced_at TIMESTAMP  
);
```

- - [] POST /api/students/:studentId/practice-session

[] Logic:

```
// When student answers question incorrectly:  
1. Identify weak concept  
2. Generate 5 similar questions (easier if struggling)  
3. Present practice session  
4. Track progress  
5. Update proficiency when 5/5 correct
```

- - **Hours 13-20:** Practice session API
 - [] GET /api/students/:studentId/weak-concepts
 - [] GET /api/students/:studentId/practice-questions
 - [] POST /api/students/:studentId/practice-answer
 - [] Track practice attempts
 - [] Update proficiency levels
 - **Hours 21-24:** Week 5 wrap-up

- [] Test entire AI pipeline end-to-end
- [] Optimize API costs (caching, batching)
- [] Demo AI features to team
- [] Document AI prompts
- [] Week 5 retrospective

Week 5 Complete! AI Features Live! 🎉🤖

- Integration Point: Adaptive learning system working ✅

WEEK 6: FACE RECOGNITION - BACKEND SUPPORT (72 hours)

Your Focus: 30 hours backend support + 42 hours other features

Note: Atharav leads face recognition implementation, you provide backend support

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Attendance Database & APIs

- **Hours 1-8:** Attendance schema

[] Create attendance tables:

```
CREATE TABLE attendance ( id UUID PRIMARY KEY, student_id UUID REFERENCES
users(id), class_id UUID REFERENCES classes(id), date DATE NOT NULL, status
VARCHAR(20), -- PRESENT, ABSENT, LATE marked_at TIMESTAMP, marked_by UUID
REFERENCES users(id), -- teacher or system method VARCHAR(50), -- MANUAL,
FACE_RECOGNITION, AUTO confidence_score DECIMAL, -- for face recognition
image_url TEXT, -- snapshot from face recognition UNIQUE(student_id, class_id,
date));CREATE TABLE classes ( id UUID PRIMARY KEY, institute_id UUID REFERENCES
institutes(id), name VARCHAR(100), batch VARCHAR(50), teacher_id UUID
REFERENCES users(id), schedule JSONB, -- {mon: "10:00-12:00", tue: "..."} created_at
TIMESTAMP);
```

○

- **Hours 9-16:** Attendance CRUD APIs
 - [] POST /api/attendance/mark (manual marking)
 - [] POST /api/attendance/mark-bulk (mark multiple students)
 - [] GET /api/attendance/class/:classId/date/:date
 - [] GET /api/students/:studentId/attendance (student's attendance history)
 - [] PATCH /api/attendance/:id (update attendance)
- **Hours 17-24:** Class management
 - [] POST /api/classes (create class)
 - [] GET /api/classes (list classes)

- [] POST /api/classes/:classId/students (add students to class)
- [] GET /api/classes/:classId/students

Deliverables Day 1-2:

- ☒ Attendance database ready
 - ☒ Manual attendance APIs working
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Face Encoding Storage & Parent Notifications

- **Hours 1-10:** Face encoding storage

[] Update students table:

```
ALTER TABLE students
ADD COLUMN face_encoding BYTEA, -- binary face encoding
ADD COLUMN face_registered BOOLEAN DEFAULT false,
ADD COLUMN face_images TEXT[]; -- array of image URLs
```

-
- [] POST /api/students/:studentId/register-face
 - Accept: Array of base64 images (5-10 photos)
 - Call Python face recognition service (Atharav builds this)
 - Store face encoding
 - Mark face_registered = true
- [] GET /api/students/:studentId/face-status
 - Return whether face is registered
 - Return number of photos captured
- **Hours 11-18:** Notification system
 - [] Set up Firebase Cloud Messaging (FCM)
 - [] Set up Twilio for SMS

[] Create notifications table:

```
CREATE TABLE notifications (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  type VARCHAR(50), -- ATTENDANCE, RESULTS, ANNOUNCEMENT
  title VARCHAR(255),
```

```
message TEXT,  
data JSONB,  
sent_at TIMESTAMP,  
read_at TIMESTAMP  
);
```

○

[] Build notification service:

```
async function sendAttendanceNotification(studentId, status) {  
  // Get student's parent  
  // Send push notification via FCM  
  // Send SMS via Twilio (if enabled)  
  // Store notification record  
}
```

○

- **Hours 19-24:** Notification APIs
 - [] POST /api/notifications/send
 - [] GET /api/users/:userId/notifications
 - [] PATCH /api/notifications/:id/read

Deliverables Day 3-4:

-  Face encoding storage ready
-  Notification system integrated

Day 5-6 (Friday-Saturday) - 24 hours

Task: Attendance Reports & Analytics

- **Hours 1-12:** Attendance reports
 - [] GET /api/attendance/reports/student/:studentId
 - Monthly attendance percentage
 - Absent dates
 - Late arrivals count
 - Trend analysis
 - [] GET /api/attendance/reports/class/:classId
 - Class-wise attendance percentage
 - Individual student breakdown
 - Low attendance alerts (< 75%)

- [] Export to PDF/Excel
- **Hours 13-20: Automated attendance tracking**
 - [] POST /api/attendance/face-recognition-batch
 - Called by face recognition service
 - Input: Array of {studentId, confidence}
 - Mark attendance for all recognized students
 - Send notifications to parents
 - [] Minimum presence logic:
 - Student must be present for 80% of class duration
 - Multiple snapshots during class
- **Hours 21-24: Testing & code review**
 - [] Test complete attendance flow
 - [] Review Atharav's face recognition code
 - [] Integration testing
 - [] Week 6 retrospective

Week 6 Complete! Face Recognition Backend Ready! 🎉📷

WEEK 7: ADVANCED PROCTORING (72 hours)

Your Focus: 36 hours backend + 36 hours coordination

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Proctoring Violation Tracking

- **Hours 1-12: Violation database**

[] Create proctoring tables:

```
CREATE TABLE proctoring_sessions ( id UUID PRIMARY KEY, attempt_id UUID
REFERENCES test_attempts(id), started_at TIMESTAMP, ended_at TIMESTAMP,
total_violations INTEGER DEFAULT 0, trust_score DECIMAL DEFAULT 100.0, -- starts at
100, decreases snapshots TEXT[] -- array of webcam snapshot URLs);CREATE TABLE
violations ( id UUID PRIMARY KEY, session_id UUID REFERENCES
proctoring_sessions(id), type VARCHAR(50), -- TAB_SWITCH, FOCUS_LOSS,
MULTIPLE_FACES, NO_FACE, SUSPICIOUS_MOVEMENT severity VARCHAR(20), --
LOW, MEDIUM, HIGH detected_at TIMESTAMP, snapshot_url TEXT, metadata JSONB --
additional details);
```

○

- **Hours 13-24:** Violation tracking APIs

- [] POST /api/proctoring/sessions (create session when test starts)
- [] POST /api/proctoring/violations (log violation from frontend)
- [] GET /api/proctoring/sessions/:sessionId

[] Violation threshold logic:

LOW: warning_count++MEDIUM: warning_count++, trust_score -= 10HIGH:

warning_count++, trust_score -= 20If warning_count >= 3 AND trust_score < 50: auto-submit test

○

- **Hours 25-36:** Real-time alerts

- [] WebSocket connection for live proctoring
- [] Send real-time alerts to teacher dashboard
- [] POST /api/proctoring/sessions/:sessionId/snapshot
 - Store webcam snapshots (every 2 minutes)
 - Upload to Supabase Storage
- [] Trust score calculation algorithm

Deliverables Day 1-3:

-  Proctoring violation tracking system
-  Real-time alerts working

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Proctoring Reports & Analysis

- **Hours 1-18:** Proctoring reports

- [] GET /api/proctoring/reports/attempt/:attemptId
 - Total violations by type
 - Timeline of violations
 - Trust score
 - Recommendation (trustworthy/suspicious/flagged)
 - Snapshot gallery
- [] GET /api/proctoring/reports/test/:testId
 - All students' trust scores
 - Flagged attempts (trust score < 70)
 - Comparison (which students had most violations)
- [] Export proctoring report as PDF

- **Hours 19-30:** Post-exam analysis
 - ☐ Machine learning model (future): Pattern recognition

☐ For now: Rule-based analysis:

If (tab_switches > 5 AND no_face_time > 2min) → FLAGGED
 If (multiple_faces_detected > 0) → FLAGGED
 If (trust_score < 60) → REVIEW_REQUIRED

- ☐ Automated flagging system
- ☐ Teacher review interface
- **Hours 31-36:** Week 7 wrap-up
 - ☐ Test entire proctoring flow
 - ☐ Performance optimization
 - ☐ Code review
 - ☐ Demo to team
 - ☐ Week 7 retrospective

Week 7 Complete! Proctoring System Ready! 🎉🔒

WEEK 8: OCR & PAPER GENERATION (72 hours)

Your Focus: 40 hours OCR backend + 32 hours coordination

Day 1-3 (Monday-Wednesday) - 36 hours

Task: OCR Integration

- **Hours 1-12:** OCR service setup
 - ☐ Research options:
 - Google Cloud Vision API (paid, accurate)
 - Tesseract OCR (free, moderate accuracy)
 - ☐ Decision: Start with Tesseract, fallback to Google Vision

☐ Set up Python FastAPI service (separate microservice):

```
# ocr_service.py
from fastapi import FastAPI, UploadFile
import pytesseract
from PIL import Image

app = FastAPI()

@app.post("/ocr/extract")
async def extract_text(file: UploadFile):
    # Read image
    image = Image.open(file.file)
    # Preprocess (deskew, denoise, binarize)
    # Run OCR
    # Return extracted text
```

○

- **Hours 13-24:** Image preprocessing

[] Implement preprocessing pipeline:

```
def preprocess_image(image):
```

```
    # 1. Convert to grayscale
    # 2. Deskew (correct rotation)
    # 3. Denoise (remove noise)
    # 4. Increase contrast
    # 5. Binarization (black & white)
    return processed_image
```

-
- [] Test with various image qualities
- [] Handle different formats (JPG, PNG, PDF)

- **Hours 25-36:** OCR API

- [] POST /api/ocr/extract-text
 - Accept image/PDF upload
 - Call Python OCR service
 - Return extracted text + confidence score
- [] POST /api/ocr/extract-with-layout
 - Not just text, but also layout (question numbers, subparts)
 - Use bounding boxes to identify structure
- [] Error handling (if OCR fails, return error with suggestions)

Deliverables Day 1-3:

- ☒ OCR service working
- ☒ Text extraction from images/PDFs

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Paper Generation Backend

- **Hours 1-12:** Text cleaning with AI
 - [] POST /api/paper/clean-text

☐ Send OCR output to Claude/GPT:

You are an expert at cleaning OCR-extracted text.

The following text was extracted from a question paper using OCR.

It may contain errors like:

- Misspelled words
- Missing punctuation
- Incorrect mathematical symbols
- Formatting issues

Clean and correct the text:

- Fix spelling errors
- Add proper punctuation
- Convert mathematical expressions to LaTeX where needed
- Maintain question numbering

OCR Text:

{ocrText}

Return corrected text.

- ☐
- ☐ ☐ Validate AI response
- ☐ ☐ Return cleaned text
- **Hours 13-24:** Document generation
 - ☐ ☐ POST /api/paper/generate-document

☐ Use python-docx to create Word document:

```
from docx import Document
from docx.shared import Pt, Inches
```

```
def generate_paper(questions, institute_name):
    doc = Document()
    # Add institute letterhead
    # Add title
    # Add instructions
    # Add questions with proper formatting
    # Add answer key (separate page)
    # Save document
    return doc_path
```

☐

- [] Support custom templates (institute letterhead)
- [] PDF generation (convert Word to PDF using LibreOffice)
- **Hours 25-36: Paper generation API**
 - [] POST /api/paper/generate-from-photo
 - Complete flow:
 1. Upload photo
 2. Run OCR
 3. Clean text with AI
 4. Detect layout
 5. Generate Word document
 6. Convert to PDF (if requested)
 7. Upload to Supabase Storage
 8. Return download link
 - [] POST /api/paper/generate-from-question-bank
 - Select questions from question bank
 - Arrange in order
 - Generate formatted paper
 - [] Manual editing support (save draft, edit, finalize)

Deliverables Day 4-6:

-  Complete paper generation flow
-  Photo to formatted paper in 2 minutes

Week 8 Complete! Month 2 Done! 🎉🎉

- Integration Point: n8n workflow can use these APIs 



MONTH 3: POLISH & LAUNCH (WEEKS 9-12)

WEEK 9: ANALYTICS & REPORTS (72 hours)

Your Focus: 40 hours implementation + 32 hours coordination

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Performance Analytics

- **Hours 1-15: Student performance analytics**
 - [] GET /api/analytics/student/:studentId/performance
 - Overall score average
 - Subject-wise performance
 - Difficulty-wise accuracy (Easy: 95%, Medium: 80%, Hard: 60%)
 - Topic-wise breakdown
 - Strong areas (90%+ accuracy)
 - Weak areas (<70% accuracy)
 - Performance trend (last 10 tests)
 - Time management analysis (fast/slow questions)
 - [] Complex SQL queries with aggregations
 - [] Caching for frequently accessed analytics
- **Hours 16-30: Comparative analytics**
 - [] GET /api/analytics/student/:studentId/comparison
 - Student score vs class average
 - Percentile ranking
 - Position in class
 - Growth rate (improvement over time)
 - [] GET /api/analytics/test/:testId/statistics
 - Average score
 - Highest/lowest scores
 - Standard deviation
 - Pass percentage
 - Question-wise accuracy
 - Time distribution
- **Hours 31-36: Predictive analytics (basic)**
 - [] Simple prediction: Based on practice test scores, predict exam score

[] Linear regression model:

// If student scored 70%, 75%, 80% in last 3 tests// Predict next test score: ~85% (upward trend)

○

- [] Recommendation system: Suggest topics to focus on

Deliverables Day 1-3:

-  Comprehensive analytics APIs

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Report Generation & Export

- **Hours 1-18:** Report generation
 - [] POST /api/reports/student/:studentId/generate
 - [] Create beautiful PDF reports using:
 - python: reportlab or WeasyPrint
 - Node.js: PDFKit or Puppeteer
 - [] Report sections:
 - Student profile
 - Overall performance summary
 - Subject-wise analysis (charts)
 - Topic-wise breakdown
 - Test history
 - Attendance summary
 - Recommendations
 - [] Customizable report templates
- **Hours 19-30:** Batch report generation
 - [] POST /api/reports/class/:classId/generate-all
 - [] Generate reports for all students in a class
 - [] Async job processing (use Bull queue)
 - [] Email reports to parents
- **Hours 31-36:** Export functionality
 - [] GET /api/reports/export/excel
 - [] Export data to Excel:
 - Student list with marks
 - Class-wise performance
 - Attendance sheets
 - [] GET /api/reports/export/pdf
 - [] Week 9 wrap-up and demo

Week 9 Complete! Analytics Ready! 🎉📊

WEEK 10: PAYMENT INTEGRATION (72 hours)

Your Focus: 40 hours implementation + 32 hours testing

Day 1-4 (Monday-Thursday) - 48 hours

Task: Razorpay Integration

- **Hours 1-12:** Setup & configuration
 - [] Sign up for Razorpay account (Test mode)
 - [] Get API keys (Key ID, Key Secret)
 - [] Install Razorpay SDK
 - [] Set up webhooks for payment events
- **Hours 13-24:** Subscription plans

[] Create subscription_plans table:

```
CREATE TABLE subscription_plans ( id UUID PRIMARY KEY, name VARCHAR(100), -- Starter, Professional, Enterprise price DECIMAL NOT NULL, duration INTEGER, -- months features JSONB, student_limit INTEGER, created_at TIMESTAMP);CREATE TABLE subscriptions ( id UUID PRIMARY KEY, institute_id UUID REFERENCES institutes(id), plan_id UUID REFERENCES subscription_plans(id), razorpay_subscription_id VARCHAR(255), status VARCHAR(50), -- ACTIVE, CANCELLED, EXPIRED start_date DATE, end_date DATE, auto_renew BOOLEAN DEFAULT true, created_at TIMESTAMP);CREATE TABLE payments ( id UUID PRIMARY KEY, subscription_id UUID REFERENCES subscriptions(id), razorpay_payment_id VARCHAR(255), razorpay_order_id VARCHAR(255), amount DECIMAL, currency VARCHAR(10) DEFAULT 'INR', status VARCHAR(50), -- SUCCESS, FAILED, PENDING payment_method VARCHAR(50), paid_at TIMESTAMP);
```

-
- **Hours 25-36:** Payment flow APIs
 - [] POST /api/subscriptions/create-order
 - Create Razorpay order
 - Return order ID for frontend
 - [] POST /api/subscriptions/verify-payment
 - Verify Razorpay signature
 - Update subscription status
 - Unlock features
 - [] POST /api/webhooks/razorpay
 - Handle payment success/failure
 - Handle subscription renewal
 - Handle subscription cancellation
- **Hours 37-48:** Subscription management
 - [] GET /api/subscriptions/plans (list all plans)
 - [] POST /api/subscriptions/subscribe (institute subscribes)
 - [] POST /api/subscriptions/upgrade (upgrade plan)
 - [] POST /api/subscriptions/cancel (cancel subscription)

- ☐ GET /api/institutes/:id/subscription (current subscription)
- ☐ Feature gates (check subscription before allowing access)

Deliverables Day 1-4:

- ☒ Razorpay integrated
 - ☒ Subscription system working
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Trial Period & Invoice Generation

- **Hours 1-12:** Trial period logic
 - ☐ 14-day free trial for new institutes
 - ☐ Trial expires → prompt to subscribe
 - ☐ Grace period (3 days) before features lock
 - ☐ Automated reminder emails (Day 10, 13, 14 of trial)
- **Hours 13-20:** Invoice generation
 - ☐ POST /api/invoices/generate
 - ☐ PDF invoice with:
 - Institute details
 - Plan details
 - Amount paid
 - GST breakdown (if applicable)
 - Payment date
 - Invoice number (unique)
 - ☐ Email invoice to institute admin
- **Hours 21-24:** Week 10 wrap-up
 - ☐ Test payment flow end-to-end
 - ☐ Test in live mode (small amount)
 - ☐ Code review
 - ☐ Demo payment system

Week 10 Complete! Payment System Live! 🎉🇮🇳

WEEK 11: PERFORMANCE OPTIMIZATION (72 hours)

Your Focus: 40 hours optimization + 32 hours review

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Database & API Optimization

- **Hours 1-12:** Database optimization

- ☐ Analyze slow queries (Supabase dashboard)

☐ Add missing indexes:

```
CREATE INDEX idx_test_attempts_student ON test_attempts(student_id);CREATE INDEX  
idx_questions_institute ON questions(institute_id);CREATE INDEX  
idx_attendance_student_date ON attendance(student_id, date);
```

-
- ☐ Optimize complex joins
- ☐ Add database constraints
- ☐ Vacuum and analyze (PostgreSQL)

- **Hours 13-24:** API response time optimization

- ☐ Measure current response times (use New Relic or custom logging)
- ☐ Target: All APIs < 500ms
- ☐ Optimization techniques:
 - Pagination (limit results to 20 per page)
 - Lazy loading (load related data only when needed)
 - Database connection pooling
 - Query result caching (Redis)

- **Hours 25-36:** Caching implementation

- ☐ Set up Redis (use Supabase or external)
- ☐ Cache frequently accessed data:
 - User sessions (already in Redis)
 - Question lists (cache for 1 hour)
 - Test configurations (cache until updated)
 - Analytics (cache for 10 minutes)
- ☐ Cache invalidation strategy:
 - Manual invalidation on update
 - TTL (Time To Live) for auto-expiry

Deliverables Day 1-3:

- ☒ Database queries optimized
- ☒ API response times < 500ms

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Security & Error Handling

- **Hours 1-12:** Security audit
 - ☐ Input validation on all APIs (Zod schemas)
 - ☐ SQL injection prevention (already safe with Supabase, but double-check)
 - ☐ XSS prevention (sanitize HTML inputs)
 - ☐ Rate limiting (100 requests/minute per user)
 - ☐ CORS configuration (allow only trusted domains)
 - ☐ Helmet.js (security headers)
- **Hours 13-24:** Error handling & logging
 - ☐ Centralized error handling middleware
 - ☐ Proper HTTP status codes:
 - 200: Success
 - 201: Created
 - 400: Bad Request (validation error)
 - 401: Unauthorized
 - 403: Forbidden
 - 404: Not Found
 - 500: Internal Server Error

☐ Structured logging (Winston):

```
logger.info("User login", {userId, timestamp});  
logger.error("Payment failed", {error, orderId});
```

- ☐
 - ☐ Log to file + external service (Logtail, Papertrail)
- **Hours 25-36:** Load testing & monitoring
 - ☐ Load test with 100 concurrent users (Apache JMeter)
 - ☐ Test critical flows:
 - 100 students starting test simultaneously
 - 50 teachers creating tests
 - 1000 question uploads
 - ☐ Set up monitoring (Sentry for errors, Uptime Robot for downtime)
 - ☐ Week 11 wrap-up

Week 11 Complete! App is Fast & Secure! 🎉🔒

WEEK 12: FINAL TESTING & LAUNCH (72 hours)

Your Focus: 40 hours deployment + 32 hours support

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Production Deployment

- **Hours 1-8:** Production environment setup
 - ☐ Set up production Supabase project
 - ☐ Migrate database schema to production
 - ☐ Configure environment variables (production API keys)
 - ☐ Set up Vercel for Next.js deployment
 - ☐ Set up Railway/Render for backend APIs
- **Hours 9-16:** Deployment
 - ☐ Deploy Next.js web app to Vercel
 - ☐ Deploy backend APIs to Railway
 - ☐ Deploy Python services (FastAPI) to Render
 - ☐ Configure custom domain (if available)
 - ☐ SSL certificate (auto-provisioned by Vercel/Railway)
- **Hours 17-24:** Database migration
 - ☐ Export sample data from dev database
 - ☐ Import into production database
 - ☐ Verify data integrity
 - ☐ Set up automated backups (daily snapshots)

Deliverables Day 1-2:

- ☒ Production environment live
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Testing & Bug Fixes

- **Hours 1-12:** End-to-end testing
 - ☐ Test complete user journeys:
 - Institute admin signs up → subscribes → creates class
 - Teacher uploads questions → creates test → publishes
 - Student registers face → takes test → views results
 - Parent receives attendance notification
 - ☐ Test on multiple devices (desktop, mobile, tablet)
 - ☐ Test on multiple browsers (Chrome, Firefox, Safari)
- **Hours 13-24:** Bug fixing

- ☐ Fix critical bugs immediately
- ☐ Document non-critical bugs (for post-launch)
- ☐ Performance testing in production
- ☐ Smoke tests

Deliverables Day 3-4:

- ☒ App is stable and bug-free
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Beta Launch & Documentation

- **Hours 1-8:** Beta institute onboarding
 - ☐ Onboard 5 beta institutes
 - ☐ Onboarding calls (demo + training)
 - ☐ Create demo accounts with sample data
 - ☐ Collect initial feedback
- **Hours 9-16:** Documentation
 - ☐ API documentation (Swagger/Postman)
 - ☐ User guides:
 - Admin guide (how to manage institute)
 - Teacher guide (how to create tests, upload questions)
 - Student guide (how to take tests, practice)
 - ☐ Video tutorials (screen recordings with voiceover)
- **Hours 17-24:** Launch preparation
 - ☐ Create landing page (marketing site)
 - ☐ Set up customer support (email, chat)
 - ☐ Prepare press release
 - ☐ Social media posts
 - ☐ Monitor production (errors, performance)

Week 12 Complete! WE LAUNCHED! 🚀 🎉

POST-LAUNCH (Month 4 and Beyond)

Your Ongoing Responsibilities:

Week 13-16 (Month 4):

1. **Bug Fixing** (20 hours/week)
 - Monitor Sentry for errors
 - Fix critical bugs within 24 hours
 - Regular releases (weekly)
2. **Performance Monitoring** (10 hours/week)
 - Check API response times
 - Database query optimization
 - Scale infrastructure if needed
3. **Customer Support** (15 hours/week)
 - Help institutes with technical issues
 - API troubleshooting
 - Feature requests collection
4. **Feature Development** (20 hours/week)
 - Build deferred features:
 - AI-based scheduling
 - Video lectures integration
 - Advanced analytics
 - Iterate based on feedback
5. **Team Management** (7 hours/week)
 - Weekly team meetings
 - Code reviews
 - Mentoring Rushikesh

Long-Term (Month 5-6):

- Scale to 100+ institutes
- Hire additional team members
- Advanced features (ML models, desktop app)
- Fundraising preparation



YOUR KEY METRICS (Track Weekly)

Development Metrics:

- [] APIs completed this week: ____ / ____
- [] Code coverage: ____% (target: 70%+)
- [] Bugs fixed: ____
- [] New bugs introduced: ____ (target: < 5)

Performance Metrics:

- [] Average API response time: ____ ms (target: < 500ms)
- [] Database query time: ____ ms (target: < 100ms)
- [] Error rate: ____% (target: < 0.1%)

Team Metrics:

- [] Code reviews completed: ____
 - [] Blockers resolved: ____
 - [] Team satisfaction: ____ / 10
-

YOUR TECH TOOLKIT

Daily Tools:

- VS Code (code editor)
- Postman (API testing)
- DBeaver/pgAdmin (database management)
- GitHub (version control)
- Supabase Dashboard (database, auth, storage)

Testing Tools:

- Jest (unit tests)
- Supertest (API tests)
- Apache JMeter (load testing)

AI Tools:

- OpenAI Playground (test prompts)
- Claude Console (test prompts)
- Postman (test AI APIs)

Monitoring Tools:

- Sentry (error tracking)
- New Relic or Datadog (performance monitoring)
- Uptime Robot (downtime alerts)

Communication:

- WhatsApp (daily standups, quick questions)
 - GitHub Issues (task management)
 - Google Meet (weekly reviews)
-

YOUR PERSONAL GOALS

Technical Skills to Master:

- Supabase (become expert)
- AI prompt engineering (advanced techniques)
- System design at scale
- Performance optimization

Leadership Skills to Develop:

- Quick decision-making
- Conflict resolution
- Mentoring junior developers
- Stakeholder communication

Career Growth:

- Build a launchable product (✅ after 3 months)
 - Lead a team successfully
 - Gain AI/ML integration experience
 - Portfolio project for future opportunities
-

FINAL CHECKLIST (Your Ownership)

By End of Week 4:

- ☐ Complete backend foundation (auth, questions, tests)
- ☐ All APIs documented
- ☐ Team B integrated with your APIs

By End of Week 8:

- ☐ AI features live (classification, generation)
- ☐ Face recognition backend ready
- ☐ OCR paper generation working

By End of Week 12:

- ☐ Production deployment complete
 - ☐ 5 beta institutes onboarded
 - ☐ App is stable and fast
 - ☐ YOU LAUNCHED A PRODUCT! 🚀
-

"I am not just a developer, I am a builder."

Every line of code you write impacts:

- Thousands of students learning better
- Hundreds of teachers working efficiently
- Dozens of institutes going digital

You're not building features. You're building dreams.

Daily Affirmation:

- "I make decisions quickly and move forward."
- "I ask for help when stuck (no ego)."
- "I support my teammates when they need me."
- "I ship quality code every single day."
- "I rest on Sundays to recharge."

Remember:

- You're the team lead, but you're also a team member
- Your team's success is your success
- Rest is not laziness, it's strategic recovery
- Done is better than perfect (for MVP)
- You got this! 💪

LET'S BUILD SOMETHING AMAZING, ADITYA! 🚀

Next Steps:

1. Read this document fully (1 hour)
2. Clarify any doubts with team (30 min)
3. Set up your development environment (Day 1)
4. START BUILDING! (Day 2)

Good luck, and I believe in you! 🙌